

PROJECT REPORT

ON

PYTHON CODE OUTPUT CHECKER

A Python-Based Application for Automatic Verification of User Code Output

Submitted By

GAYATRI SABLE

Diploma in Artificial Intelligence and Machine Learning

Project Overview

The Python Code Output Checker is a graphical user interface (GUI) application developed using Python and Tkinter. The application enables users to solve programming problems by writing Python code and automatically checks whether the generated output matches the expected output. The system provides instant feedback and helps users improve their coding, logical thinking, and problem-solving skills through interactive practice.

Technologies Used

- Python
- Tkinter
- sys Module
- io Module

"A GUI-Based Python Application for Code Output Verification"

Introduction

Python Code Output Checker is a GUI-based application developed using Python and Tkinter. The application allows users to enter Python code for a given programming problem and automatically checks whether the generated output matches the expected output. It provides instant feedback and helps beginners practice programming in an interactive manner.

Objective

The main objective of this project is to develop an application that automatically verifies the output generated by user-written Python code. The project aims to help users improve their coding skills by providing immediate feedback on their solutions.

Problem Statement

Beginners often face difficulties in verifying whether their program outputs are correct. This project provides a simple platform where users can write Python code, execute it, and compare the generated output with the expected result automatically.

Technologies Used

- Python
- Tkinter
- io Module
- sys Module

Features

- User-friendly graphical interface.
- Displays programming questions one by one.
- Accepts Python code as input.
- Executes the entered code automatically.
- Captures and evaluates program output.
- Compares generated output with expected output.
- Provides instant feedback.
- Automatically moves to the next question.
- Displays completion message after all questions are solved.

Working of the Project

1. The application displays a programming problem.
2. The user enters Python code in the text area.
3. When the Submit button is clicked, the program executes the code.
4. The output generated by the user's code is captured.
5. The captured output is compared with the expected output.
6. If the outputs match, a "Correct" message is displayed.

7. If the outputs do not match, the application displays both the user's output and the expected output.
8. The next problem is loaded automatically.
9. After completing all problems, a completion message is displayed.

Sample Problems

1. Print "Hello World"
2. Print numbers 1, 2, 3 (each on a new line)
3. Print all elements of [3, 6, 9]

Program Code

```
import tkinter as tk
from tkinter import messagebox
import sys
import io

# Questions
problems = [
    ("Print 'Hello World'", "Hello World"),
    ("Print numbers 1, 2, 3 (each on a new line)", "1\n2\n3"),
    ("Print all elements of [3, 6, 9]", "3\n6\n9")
]

current = 0 # Start with first problem

def check_code():
    global current

    code = code_box.get("1.0", tk.END)

    # Capture output
    old_stdout = sys.stdout
    sys.stdout = io.StringIO()

    try:
        exec(code, {})
        output = sys.stdout.getvalue().strip()
    except Exception as e:
        output = f"Error: {e}"

    sys.stdout = old_stdout

    # Expected output
    expected = problems[current][1]

    # Check result
    if output == expected:
```

```

        messagebox.showinfo("Result", "✔ Correct!")
    else:
        messagebox.showinfo(
            "Result",
            f"✘ Wrong!\n\nYour Output:\n{output}\n\nExpected:\n{expected}"
        )

    # Next Problem
    current += 1

    if current < len(problems):
        show_problem()
    else:
        messagebox.showinfo("Done", "🎉 All problems completed!")
        root.destroy()

def show_problem():
    problem_label.config(
        text=f"Problem {current + 1}: {problems[current][0]}"
    )
    code_box.delete("1.0", tk.END)

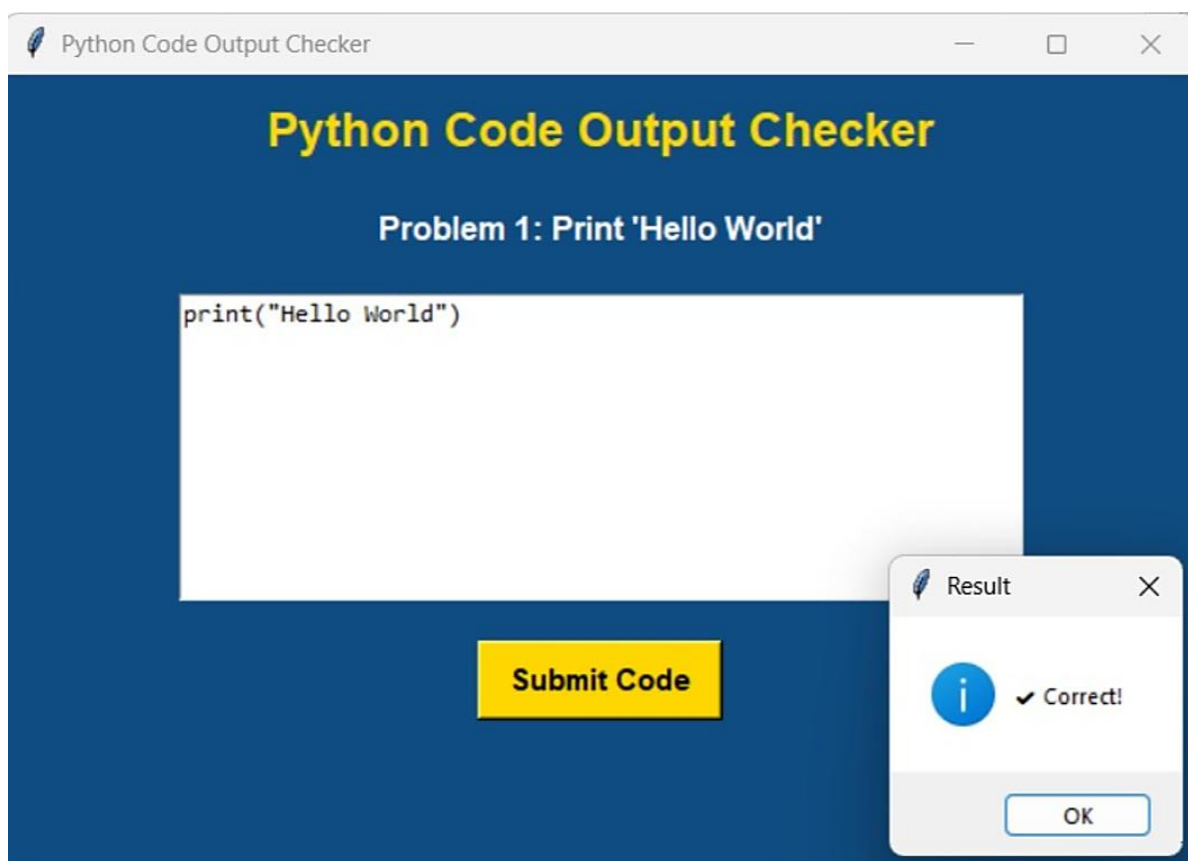
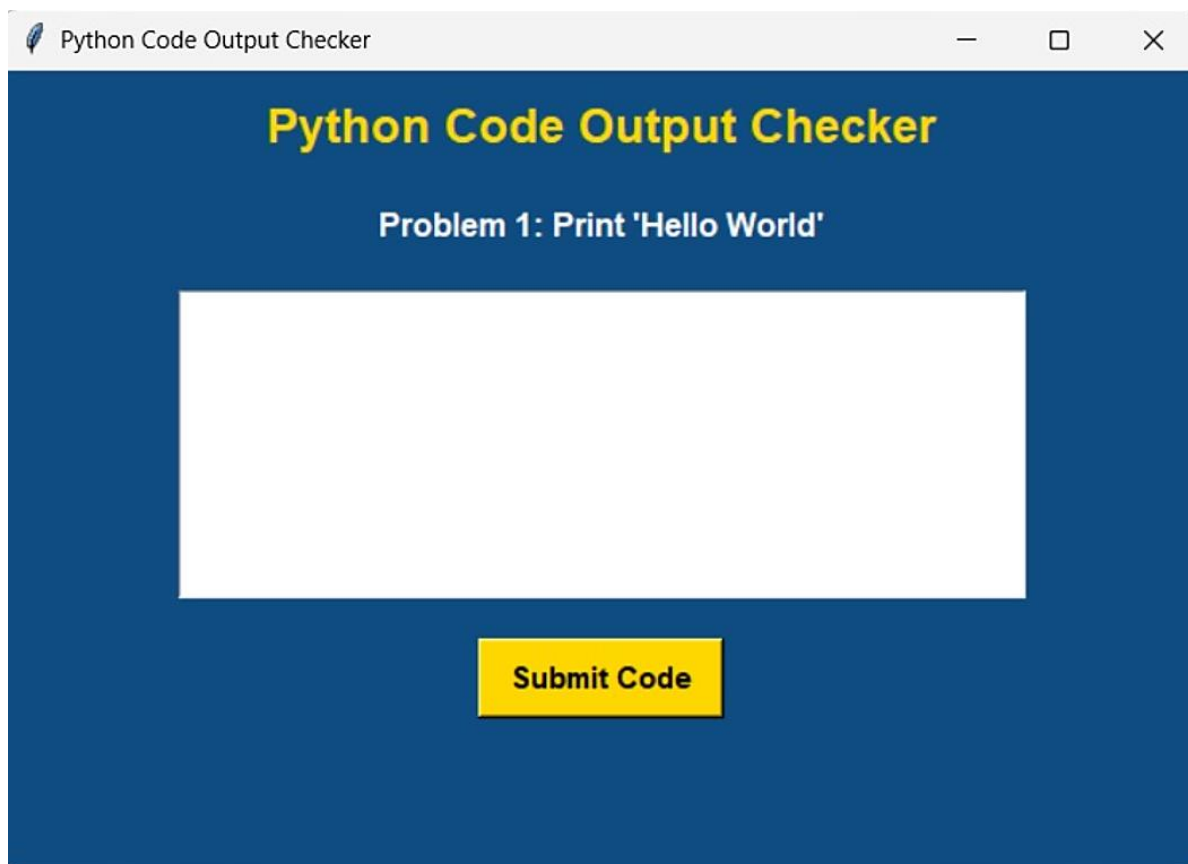
# GUI
root = tk.Tk()
root.title("Code Output Checker")
root.geometry("500x350")
root.configure(bg="#FFB6C1")

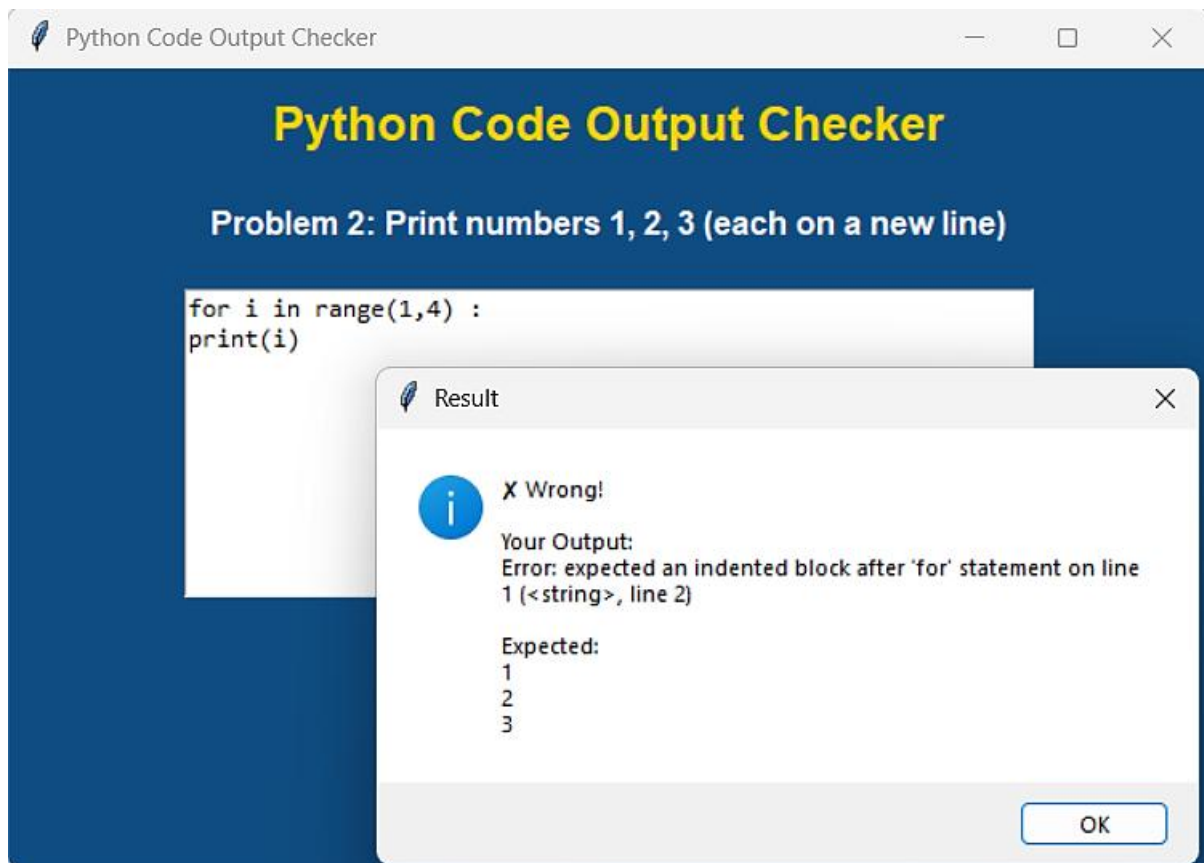
problem_label = tk.Label(
    root,
    text="",
    font=("Arial", 11),
    bg="#FFB6C1"
)
problem_label.pack(pady=10)

code_box = tk.Text(root, height=8, width=50)
code_box.pack(pady=10)
submit_button = tk.Button(
    root,
    text="Submit Code",
    command=check_code
)
submit_button.pack(pady=5)
show_problem()
root.mainloop()

```

Output





Advantages

- Helps beginners practice Python programming.
- Provides immediate feedback.
- Easy to use and understand.
- Improves logical thinking and problem-solving skills.
- Interactive learning environment.

Future Scope

- Add more programming questions.
- Introduce multiple difficulty levels.
- Maintain score records.
- Add timer functionality.
- Support multiple programming languages.
- Store user progress and performance reports.

Conclusion

The Python Code Output Checker successfully evaluates user-written Python code by comparing the generated output with the expected result. The project provides instant feedback and creates an interactive learning environment for beginners. It helps users improve their programming skills and understand coding concepts more effectively.